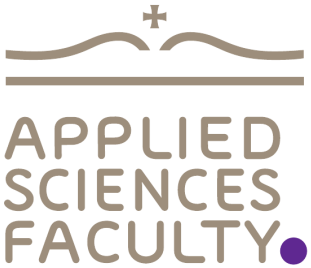
C++ — лекція 3. Інструкції керування потоком виконання





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

Інструкція вибору — if

Один із способів це зробити — оператор if:

```
if( вираз-умова )
    Вираз;
```

Де "вираз" може бути практично будь-яким, зокрема — оператором порівняння. Також у ньому можна оголошувати змінні, які будуть існувати до кінця if (if-else). "*оператор*" виконуватиметься тільки, якщо "вираз" поверне true. Оператор може бути і складеним — блоком, виділеним фігурними дужками:

```
if( вираз-умова )
{
    Вираз-1;
    Вираз-2;
    ........
    Вираз-N;
}
```

if-else

```
if( вираз-умова )
    Вираз-1;
else
    Вираз-2;
```

```
if( вираз-умова )
{
    Вираз-1-1;
    Вираз-1-2;
    ........
    Вираз-1-N;
} else
{
    Вираз-2-1;
    Вираз-2-2;
    ........
    Вираз-2-N;
}
```

Оголошення змінних в if – C++17

```
int get_handler(const std::string& fnm);

      ......................

if(auto handler = get_handler("test"); handler != -1)
{
        cout << "File opened" << endl;
            ..................
}else{
        cerr << "Failed to open file" << endl;
            ..................
}
```

- Змінна, так створена, видима і в if і в else, але не видима за їх межами.
- C++20 додав можливість використання typedef, а C++23 – і using:

```
for (typedef int T; T e : v) // C++20

for (using T = int; T e : v) // C++23
```

if constexpr – C++17

```
template <typename T>
auto get_value(T t) {
    if constexpr (std::is_pointer_v<T>)
        return *t;
    else
        return t;
}
```

- if constexpr обчислюється під час компіляції. Фактично, це засіб метапрограмування.
- В загальному випадку, вираз повинен бути коректним, навіть якщо викидається.
- Детальніше про нього – пізніше.

if consteval – C++23

- C++23 додав `if consteval { ... }`, без умови, тіло якого виконується, лише якщо обчислення відбуваються під час компіляції[1].
  - ◇ Еквівалентний `if (std::is_constant_evaluated()) { ... }`.
  - ◇ У ньому можна викликати consteval-функції (на відміну від constexpr-контексту).
- Окрім else, підтримується ще така форма: `if ! consteval { }`, можливо – зі своїм else.
- Детальніше див. `https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p1938r3.html#introduction`.

[1] Чи, більш строго – оскільки стандарт не виділяє етап компіляції та етап виконання, під час constant evaluation.

Інструкція вибору — switch

```
switch ( expression )
{
  case constant1:
     group of statements 1;
     break;
  case constant2:
     group of statements 2;
     break;

     .....

  default:
     default_statements;
}
```

```
bool exit_program = false;
char responce;
cin >> responce;
switch(responce)
{
case 'a':
      action_for_a();
     break;
case 'B':
     action_for_B();
     break;
default:
     cout << "Unknown code";
case 'x':
case 'X':
     exit_program=true;
     break; // Завершуємо switch
};
if (exit_program)
      return 0;
```

- Оголошення змінних в case заборонене[2]:

```
switch(1) {
    case 1: int x = 0; //  initialization
            std::cout << x << '\n';
            break;
    default: // compilation error: jump to default:
             // would enter the scope of 'x' without initializing it
             std::cout << "default\n";
}
```

- Тому оголошення потрібно включати в локальну область видимості:

```
    case 1: {
            int x = 0; //  initialization
            std::cout << x << '\n';
            break;
            }
```

- В default змінні оголошувати можна[3].

[2] Причина – реалізація switch як набору goto, при єдиній області видимості switch, див. відповідний розділ. Тому, якщо ви оголошуєте змінну фундаментального типу без ініціалізації, воно може й скомпілюватися. Але, все рівно – погана ідея. Ця примітивна структура switch ще й в інших місцях проглядає – не зловживайте нею.

[3] Так як її ініціалізацію вже ніхто не перестрибне.

Покращення switch в C++17

Оголошення змінних:

```
switch(auto handler = file("test"); handler.get_handler()) {
case 0:
    cout << "Stdin" << endl; break;
case 1:
    cout << "Stdout" << endl; break;
case 2:
    cout << "Stderr" << endl; break;
default:
    cout << "Other" << endl;
}
```

Атрибути:

```
switch(responce)
{
case 'x':
        [[ fallthrough ]]
case 'X':
    exit_program=true;
```

Тернарний оператор вибору

Єдиний оператор, що працює зразу з трьома операндами.

`(вираз_умова) ? (вираз1) : (вираз2)`

Іноді може бути зручною заміною конструкції if-else.

Приклад — надати змінній c більшу із a і b:

```
c = a>b ? a : b;
```

while

```
while( виразумова_ )
    Вираз;
while( виразумова_ )
{
    Вираз—1;
    Вираз—2;
     .....
    Вираз—N;
}
```

- Вираз, який може бути складеним, буде повторюватися доти, поки умова "вираз_умова" є істинним, true.
- В умові можна оголошувати змінну: `while(Type t = get_cond()){...}`. При тому, вона буде створюватися наново на початку кожної ітерації та знищуватися в кінці. При цьому умова буде еквівалентна if(t) – виконуємо тіло.
- Щоб виконання циклу могло завершитися, умова циклу повинна у ньому[4] змінюватися.
- Міркування, чому додали створення змінних окремо від перевірки умови в if, for, switch, але не в while: `https://quuxplusone.github.io/blog/2020/10/28/while-with-initializer/`.

[4] Або, у випадку багатопотокового програмного забезпечення, чи п.з., яке працює з апаратурою – ззовні. Цитата з `https://en.cppreference.com/w/cpp/language/while`: "*As part of the C++ forward progress guarantee, the behavior is undefined if a loop that has no observable behavior (does not make calls to I/O functions, access volatile objects, or perform atomic or synchronization operations) does not terminate. Compilers are permitted to remove such loops.*"

do ... while

- У випадку, якщо є потреба спочатку один раз виконати тіло циклу, а потім вже перевіряти умову:

```
do
    Вираз;
while( виразумова_ );

do
{
    Вираз–1;
    Вираз–2;
     ....
    Вираз–N;
}
while( виразумова_ );
```

- Спочатку виконується оператор, аж потім перевіряється умова.

for

forініціалізація(<>; умова<завершення_>; зміна<лічильника_>)
    вираз<тілоциклу__>;

Наприклад:

```
for(int i=0; i<10; ++i)
{
    cout << i << endl;
}
```

- <ініціалізація> задає початкові умови циклу. В нашому прикладі — i=0. Може містити оголошення змінних[5]. Такі змінні існуватимуть до кінця циклу.
- <умова_завершення> вказує, за виконання якої умови слід продовжувати виконання циклу. Наприклад, продовжувати, поки i>10.
- <зміна_лічильника> виконуватиметься на кожній ітерації циклу. Природне використання — змінювати лічильник.

[5] В C++ та новіших версіях C.

Range for – C++11

- В C++11 з'явилася спеціалізована форма циклу for, для обходу тих чи інших послідовностей (інтервалів) поелементно — один елемент за раз[6].
- Наприклад, вивести всі елементи контейнера можна так:

```
vector< double > v1;
     .....................
for ( double x : v1) {
     cout << x << endl;
}
```

- Звичайно, зручніше писати так:

```
for ( auto x : v1) {
     cout << x << endl;
}
```

- Вміє працювати із вбудованими масивами:

```
int ba[15] = {1,2,3};
for ( auto x : ba){
     cout << x << endl;
}
```

[6] Тобто, цей цикли свідомо зроблено простим. Якщо потрібно складнішу обробку, наприклад таку, яка включає одночасну роботу із кількома елементами того ж масиву, слід скористатися більш загальним циклом.

for для інтервалів — range for — 2

- Може використовуватися для зміни елементів:

```
for ( auto& x : v1){
    x = rand() % 100;
}
```

- Як видно, по замовчуванню елементи в x копіюватимуться. Якщо змінювати їх не потрібно, але і копіювати теж – дорого, можна скористатися звичною ідіомою передачі за константним посиланням:

```
for ( const auto& x : v1){
    process_data(x);
}
```

- Те ж стосується модифікатора volatile.
- Вміє працювати із:
  - Всіма контейнерами стандартної бібліотеки C++, включаючи array і valarray.
  - Списками ініціалізації.
  - Співпадіннями регулярних виразів (regular expression matches).
  - **Будь-якими користувацькими типами, для яких функції begin(T) і end(T) повертають ітератор для обходу цих типів**.

Уявлення про роботу range for

- Цикл:

```
for ( var : expression ) {
  statement1;
    ..........
}
```

приблизно еквівалентний:

```
{
  auto&& range = expression;
  for(auto itr = begin(range), fin = end(range); itr != fin; ++itr ) {
    var = *itr;

    statement1;
      ..........
  }
}
```

- Ремарка: код, який робить посилання на змінну циклу[7] може поводитися неочікувано – вона є посиланням на елемент масиву, яке перепризначається.

[7] Наприклад, у лямбда-функції захоплює її за посиланням і кудись передається

Range for – C++17

- Починаючи з C++17, можна використовувати

```
for (auto&& [first,second] : mymap) {
                    .....
}
```

  - Елементами mymap є той чи інший кортеж з двох елементів (ймовірно, pair<T, U>).
  - && тут означає створення rvalue reference, ми вивчатимемо його пізніше, якщо коротко – це спосіб уникнути зайвих копіювань у загальному коді – який мало що знає про тип своїх аргументів.

- В C++17 range for поводиться так (назви змінних – умовні):

```
{
    auto && __range = виразінтервалу_ ;
    auto __begin = визначенняпочаткуінтервалу__;
    auto __end = визначеннякінцяінтервалу__;
    for ( ; __begin != __end; ++__begin) {
        елементінтервалу_ = *__begin;
        тілоциклу_
    }
}
```

- Визначення інтервалу, якщо коротко, відбувається так:
  - ◇ Якщо аргумент – масив C, __begin – вказівник на його початок, __end – вказівник на початок плюс розмір – вказівник на кінець масиву, умовно – перший відсутній елемент.
  - ◇ Якщо вираз_інтервалу повертає щось, що має методи begin() та end(), використовуються вони: __begin = вираз_інтервалу.begin(), __end = вираз_інтервалу.end().
  - ◇ Якщо ж таких методів немає, використовуються функції begin() та end(): __begin = begin(вираз_інтервалу), __end = end(вираз_інтервалу).
- До C++17 __begin та __end обчислювалися на кожній ітерації.
- Починаючи з C++17 вони можуть мати різний тип. Це зручно, коли кінець визначається предикатом, наприклад – у C-стрічці.

Range for – C++20

- C++ додав оголошення змінних в циклі.
- Ініціалізація виконується один раз, перед визначенням початку та кінця і є локальною для циклу.
- ◇ Приклад:

```
for (Type data = get_data(); auto& item : data.items()) {
                    ..............................
}
```

- Зауважте, це – помилка, якщо get_data() повертає копію:

```
for (auto& item : get_data().items()) {
                    ..............................
}
```

- ◇ Причина – якщо час життя тимчасового значення у виразі з інтервалом:

```
for (auto& item : get_data())     .............
```

буде розширено на час виконання циклу, то існування тимчасових величин із його підвиразів (об'єкт, повернений get_data() в get_data().items()) – ні.

- Детальніше див. `https://en.cppreference.com/w/cpp/language/range-for`.
- В C++23 проблему вирішили системніше – всі об'єкти, створені після :, існуватимуть до кінця циклу: `wg21.link/P2644`, `wg21.link/p2718`.

Команди break і continue

- Іноді цикл слід перервати передчасно. Це можна зробити за допомогою інструкції **break**.
- Інструкція **continue** пропускає всі оператори до кінця тіла циклу і зразу переходить до наступної ітерації.

Інше

- В деяких інших мовах потреба в goto сильно зменшена, завдяки можливістю зробити break чи continue для конкретного циклу.
  - ◇ В C++ такого поки немає, але в C – недавно було запропоновано в 2024.
  - ◇ Аналіз пропозиції, хоча з критичної до запропонованого синтаксису, див.: "Labeled loop syntax in many languages" та ""Cumbersome" labeled loops and the C preprocessor".
- Цікавий філософський есей: "Structured iteration".

goto

- Вираз/інструкція goto — безумовний перехід.

```
        goto label;
        cout << "Skipped." << endl;
label:
        cout << "Resumed execution here." << endl;
```

- **Категорично** не рекомендується до використання. Важливі винятки з цього правила:
  - ◇ Автоматично згенерований код, з яким людина не повинна працювати може собі дозволити – часто такий простіше генерувати.
  - ◇ Розумним є вихід із вкладених циклів чи пропуск ітерації зовнішнього циклу[8]. Альтернативи, типу перевірки прапорця умови виходу в циклах, що охоплюють даний, зазвичай, ще гірші, ніж goto.
  - ◇ Хоча деякі Guidelines рекомендують мати одну точку виходу з функції, до якої стрибати goto, в цілому це вважається небажаною практикою[9].

[8] Java підтримує для цього break і continue з міткою, в C++ доводиться вручну

[9] Метафора: навіть якщо хірург перед операцією мусить 5 хв. мити руки, не означає що так потрібно робити в більшості інших контекстів.

Обмеження goto

- Стрибати можна лише в межах однієї функції.
- Якщо goto покидає область видимості локальних змінних, вони знищуються звичним способом – з викликом деструкторів.
- В try-блок чи catch-блок стрибати не можна, але можна стрибати із них.
- Не можна стрибати після ініціалізації змінних (включаючи неявну ініціалізацію конструктором по замовчуванню) у якомусь блоці.
  - Але пропускати створення неініціалізованих змінних можна.
  - Див. також поведінку switch() – вона базується на ледь замаскованих goto.

